PROGETTAZIONE DEL SOFTWARE SICURO

Lezione 7

Dai Modelli al Codice
Vulnerabilità del codice
Meccanismi di sicurezza in Java

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Argomenti della lezione

- **Da FSM a codice Java**
- **Vulnerabilità in codice C**
- **Meccanisma di sicurezza in Java e Java sandbox**

MACCHINE A STATI FINITI IN JAVA

Ripasso FSM

Già viste nell'unità didattica "Macchine a stati finiti"

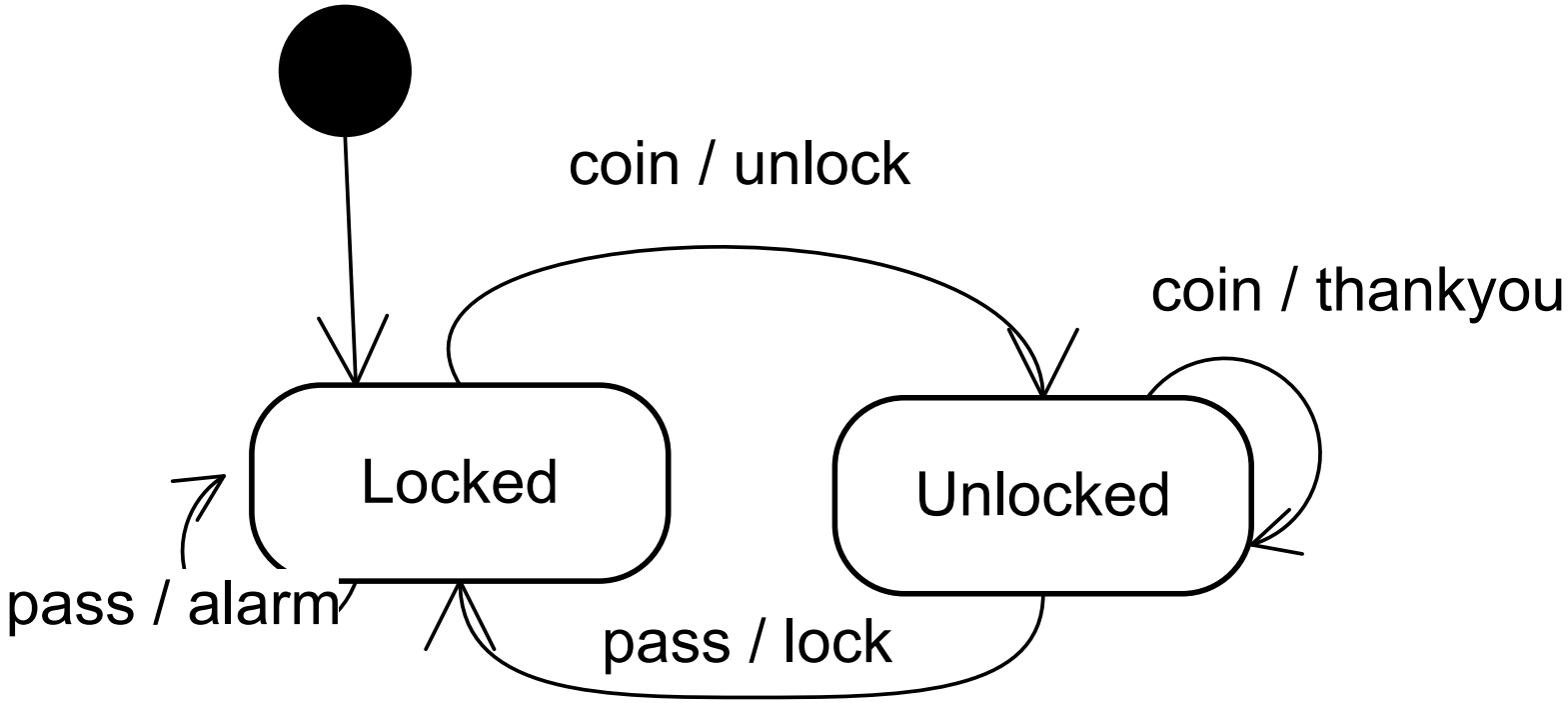
Usate spesso per modellare e simulare hw e sistemi di controllo

Sono:

- un insieme di stati
- con transizioni tra stati
  - la transizione è determinata dall'input (evento) e dallo stato corrente
  - ogni transizione può avere un output (azione)

Come si possono tradurre in Java ?

Un sistema Turnstile (tornello)



Pattern del software

Un *pattern del software*

- È una **soluzione** provata e applicabile a un particolare problema di progettazione
- Il pattern descrive la soluzione in una **forma standard**, in modo che possa essere facilmente condivisa e riusata

Una possibile descrizione strutturata

- nome
- contesto - la situazione in cui il pattern può essere applicato
- problema
- soluzione
- conseguenze – risultati (positivi e negativi) e compromessi

Design pattern

Un design pattern [GoF]

- è la descrizione di oggetti e classi che comunicano, personalizzati per risolvere un problema generale di progettazione in un contesto particolare

I design pattern vengono usati nell'ambito della progettazione di dettaglio di componenti

Esempi di design pattern

- Singleton, State, Adapter, Proxy, Observer, ...

State Pattern - definizione

Scopo:

- Permettere ad un oggetto di cambiare il suo comportamento quando il suo stato interno cambia.

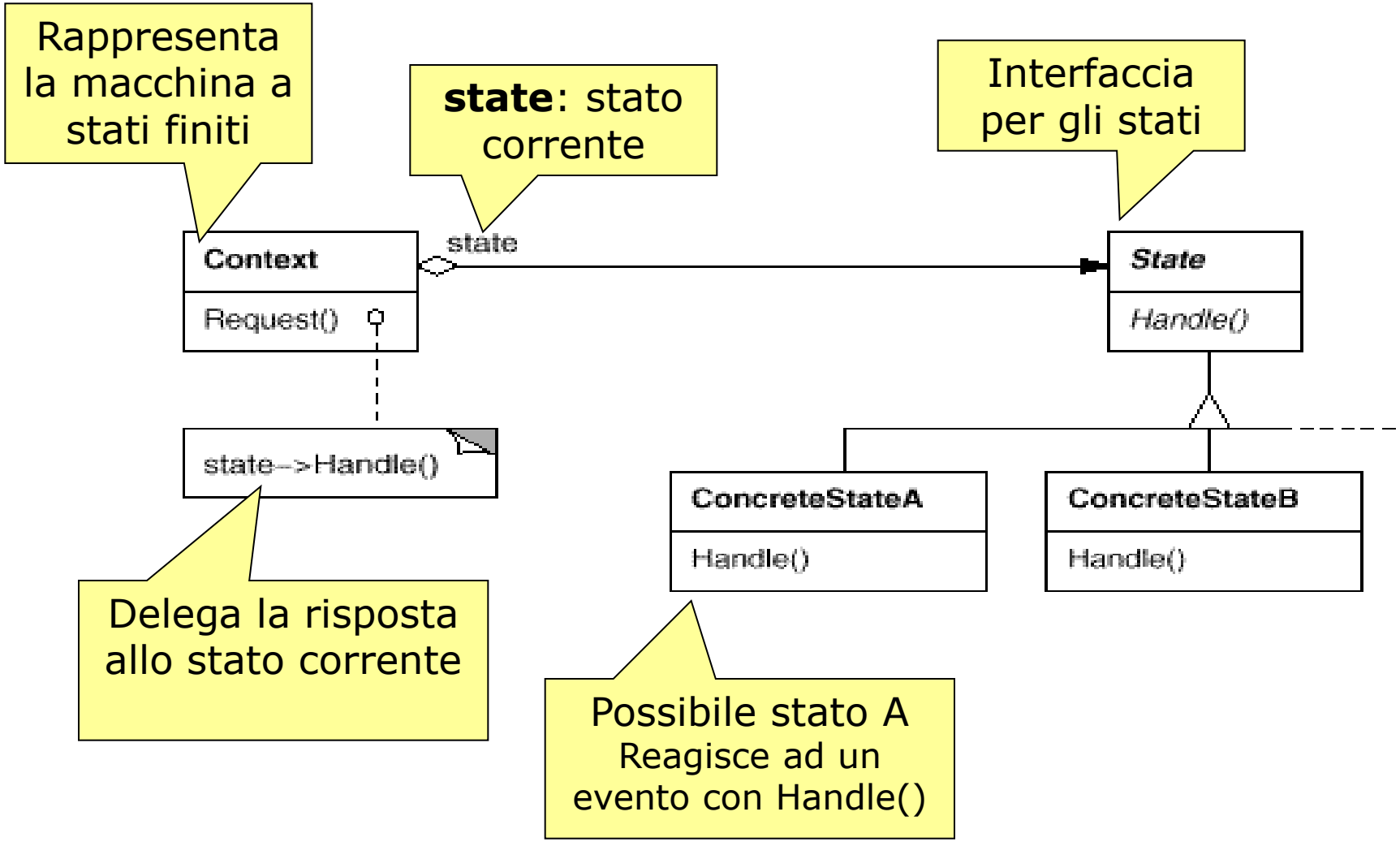
Contesto in cui si applica:

- Quando il comportamento di un oggetto dipende da un suo stato. Quando si esprime tale comportamento come una istruzione condizionale sullo stato e sull'evento ricevuto.

Soluzione proposta:

- Definisci una classe concreta per ogni stato. Le classi di stato implementano una interfaccia comune che reagisce ad ogni possibile input. Gli oggetti di stato possono richiamare azioni sull'oggetto che li controlla e decidono loro il prossimo stato (cioè le transizioni di stato).

State Pattern - schema delle classi



State Pattern

Crea una gerarchia di classi per gli stati

- Con una interfaccia che modella tutte le possibili reazioni agli input

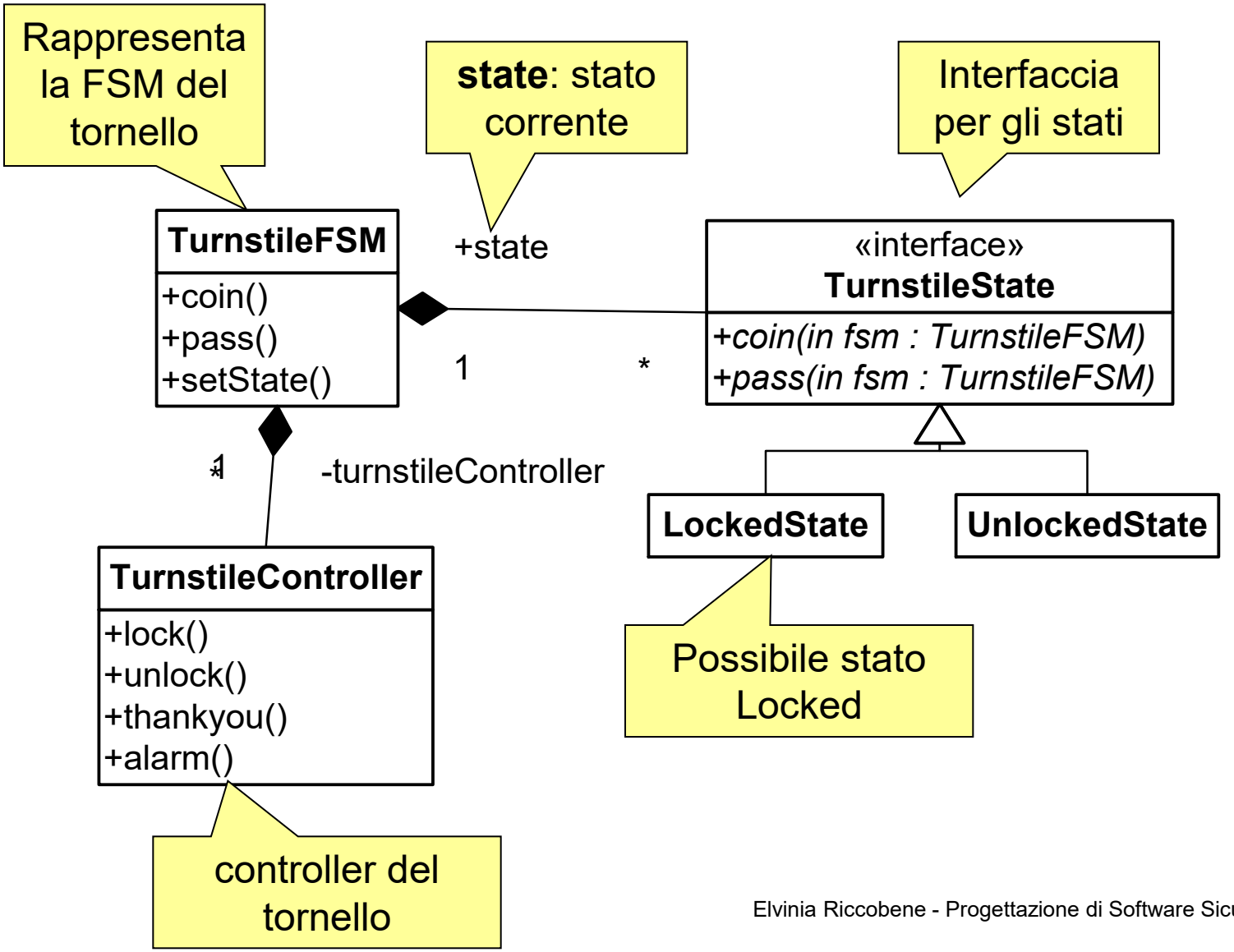
L'oggetto che rappresenta la FSM punta ad una istanza (in modo polimorfico) di una classe che rappresenta lo stato corrente

- Quando lo stato cambia, punterà ad una istanza diversa (ma sempre di tipo stato)

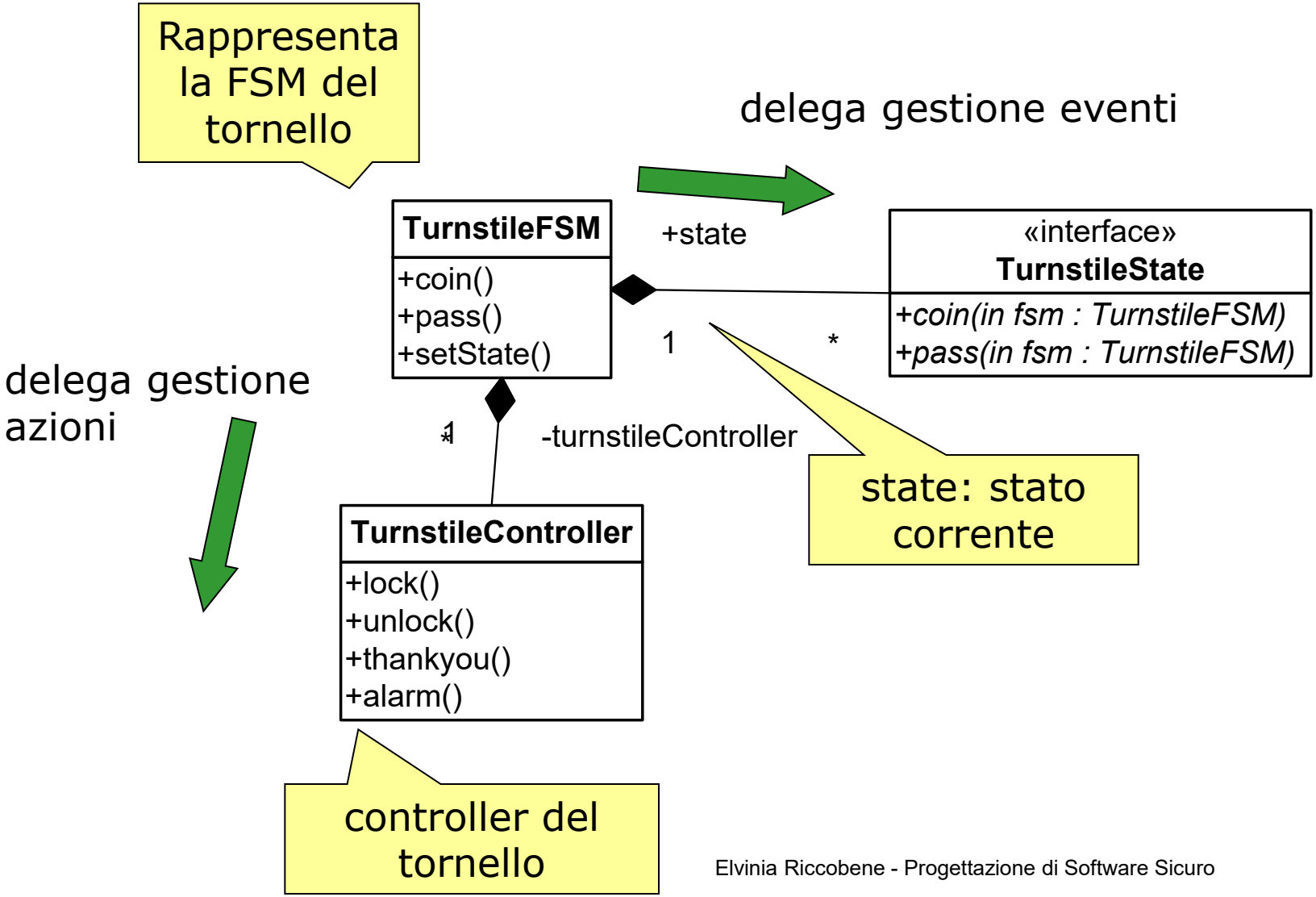
Lo stato successivo viene deciso dall'oggetto che rappresenta lo stato corrente

- Più modulare (ma anche più codice)
- Il codice è più pulito
- L'estensione non comporta di modificare il codice esistente

Turnstile - schema classi



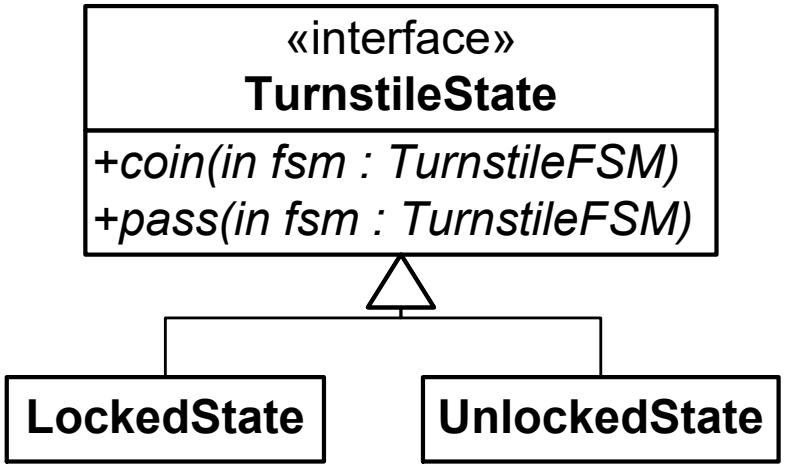
TurnstileFSM - schema



Classe TurnstileFSM

```
/* rappresenta la FSM del tornello */
class TurnstileFSM {
  // il controllore del tornello
  private TurnstileController turnstileController;
  // lo stato attuale
  private TurnstileState state = …
  // delega allo stato corrente l'evento
  public void coin() { state.coin(this);}
  public void pass() { state.pass(this);}
  // agisci sul controller quando richiesta azione
  void thankyou() {turnstileController.thankyou();}
  void alarm() {turnstileController.alarm();}
  void lock() {…}
  void unlock() {…}
```

La gerarchia di stati



ogni sottoclasse implementa TurnstileState e definisce un oggetto costante che rappresenta quello stato (singoletto – singleton pattern):

```
class LockedState implements TurnstileState {

      public static TurnstileState LOCKEDSTATE

            = new LockedState();

...
```

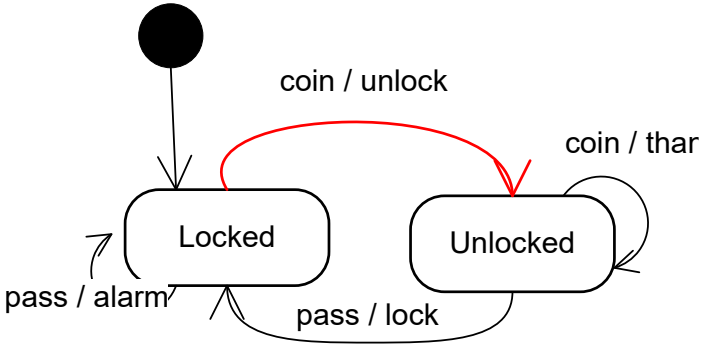
Classi TurnstileState

```
/* rappresenta lo stato del tornello*/
interface TurnstileState {
  void coin(TurnstileFSM t);
  void pass(TurnstileFSM t);
}
/* rappresenta lo stato LockedState */
class LockedState implements TurnstileState {
  // dice cosa accade quando il tornello è locked
  // e accade l'evento coin
  public void coin(TurnstileFSM t) {…}
  // idem quando pass
  public void pass(TurnstileFSM t) {…}
}
/* rappresenta lo stato UnLockedState */
class UnlockedState implements TurnstileState {…}
```

Singolo metodo in stato

Il cambiamento di stato e l'azione al verificarsi dell'evento e è delegato al metodo ed dello stato corrente

Esempio: evento coin in LockedState:



per azionare la fsm devo passarla come parametro

```
class LockedState …
   public void coin(TurnstileFSM t) {
    t.setState(UnlockedState.UNLOCKEDSTATE);
    t.unlock();
}
```

Aggiunta di uno stato/ evento

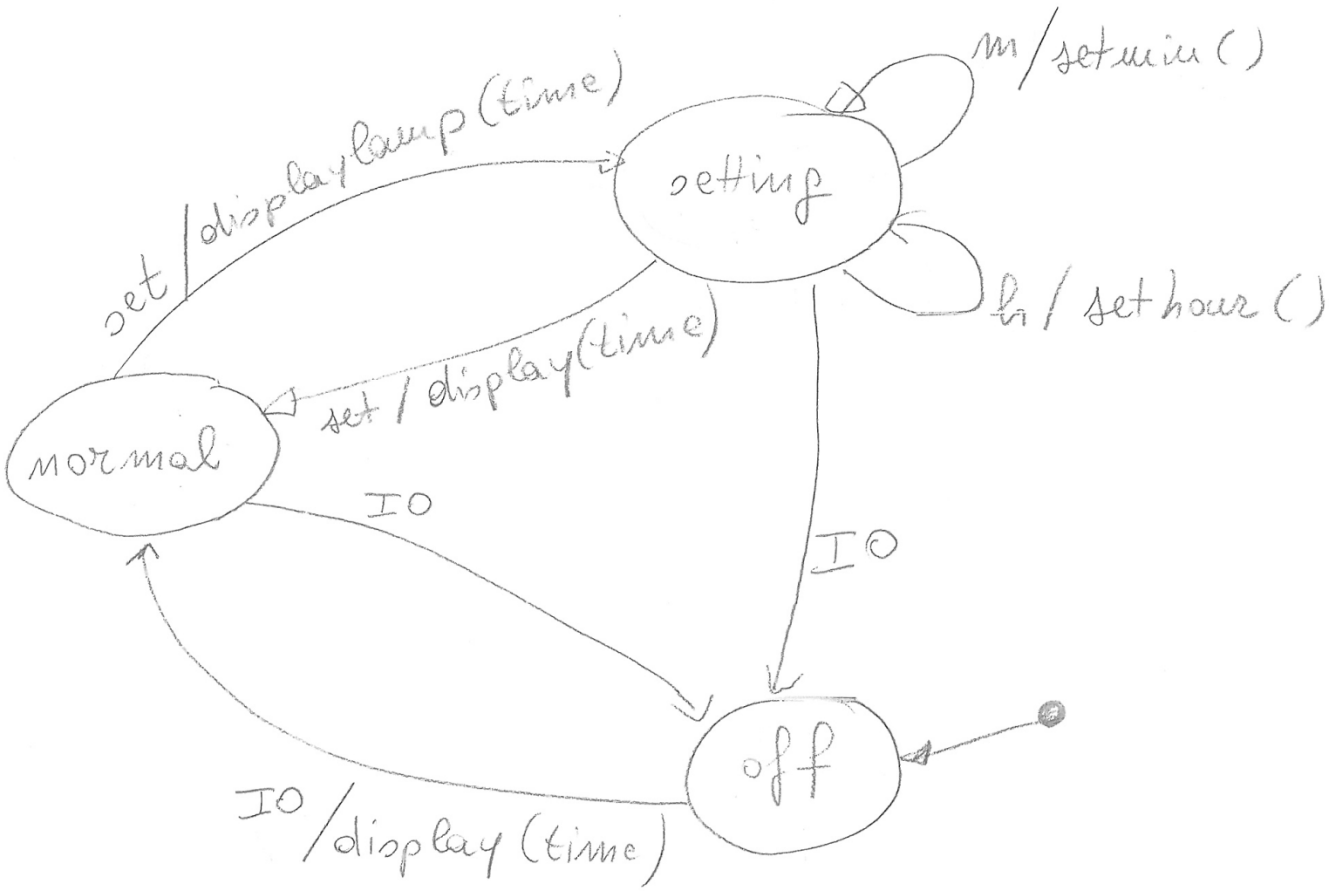
se voglio aggiungere uno stato NS:

- aggiungo una sottoclasse di stato NS che implementerà i metodi *coin*() e *pass*()e definirà una costante NS

se voglio aggiungere uno evento EV

- aggiungo un metodo EV all'interfaccia classe TurnstileState
- tutti gli stati dovranno definire cosa fare quando accade EV

FSM orologio digitale

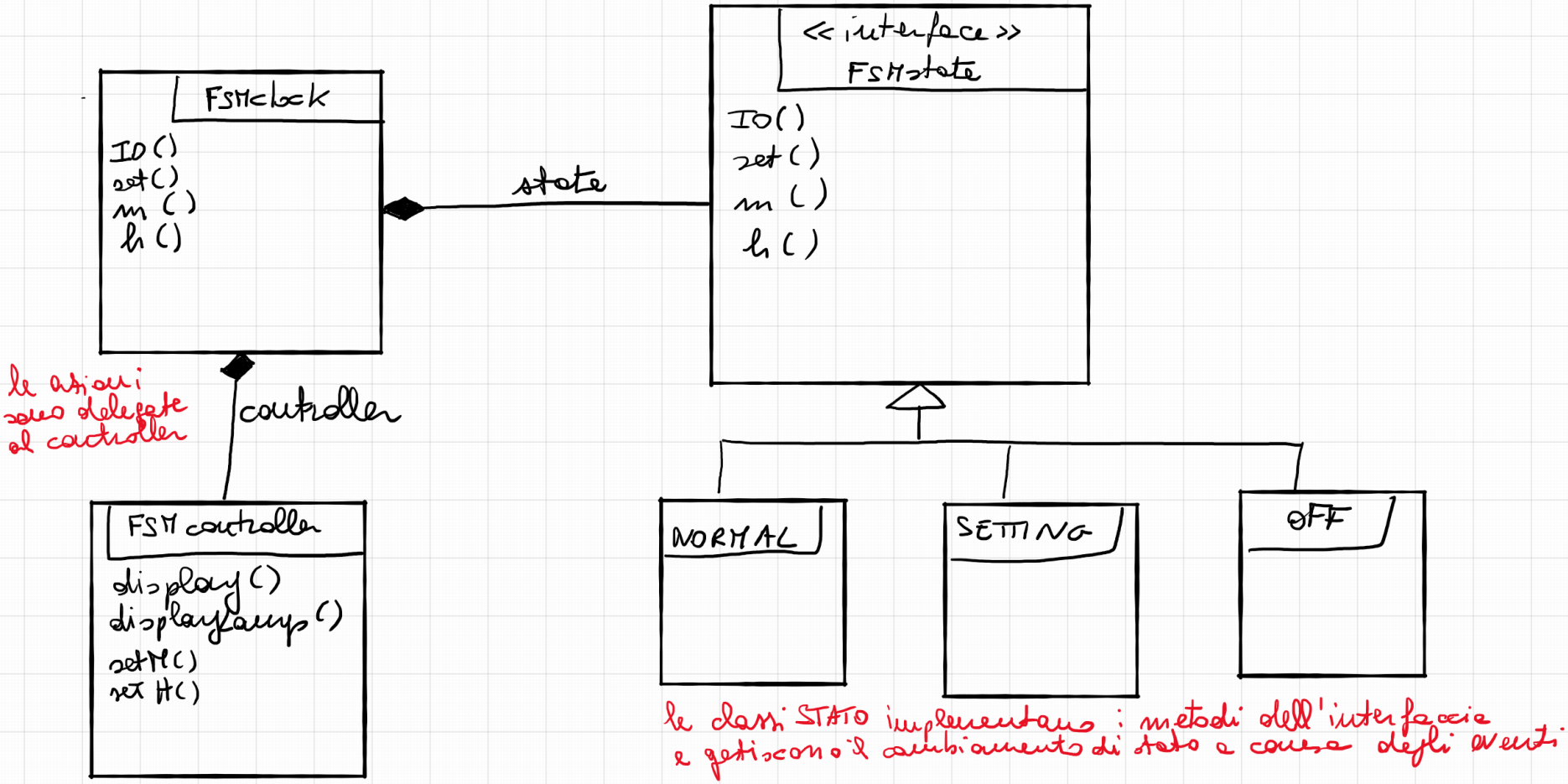


3 STATI

I = { IO(), set(), m(), h() }

AZIONI = { display(), displaylamp(), setmin(), sethour() }

Istanziare lo state pattern



Codifica su Ariel

Implementare in Java una FSM usando lo state pattern

In sintesi:

- la FSM è rappresentata da una classe che delega le azioni ad un controller e le transizioni di stato a una gerarchia di stati
- ogni possibile stato è una sottoclasse di una interfaccia e implementa un metodo per ogni evento – input possibile
- nel metodo del singolo stato c'è il codice che gestisce il cambio di stato e l'azione – output quando la FSM si trova in quello stato

Tool automatici

Esistono numerosi tools che permettono di disegnare (o scrivere) la FSM e poi derivano loro tutto il codice:

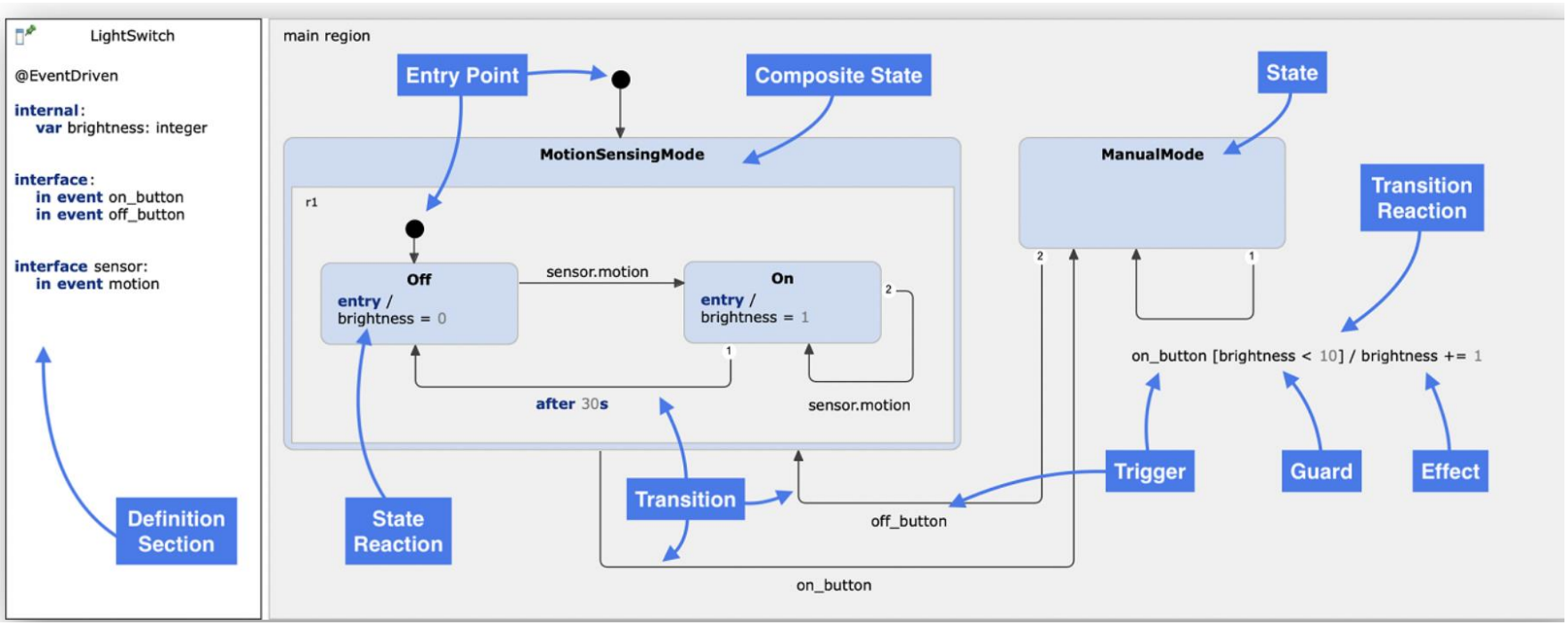
- SMC *The State Machine Compiler*

Prende una descrizione testuale della macchina e produce codice C, C++, Java, php , …

Il più performante:

YAKINDU Statechart tools

Una FSM in Yakindu



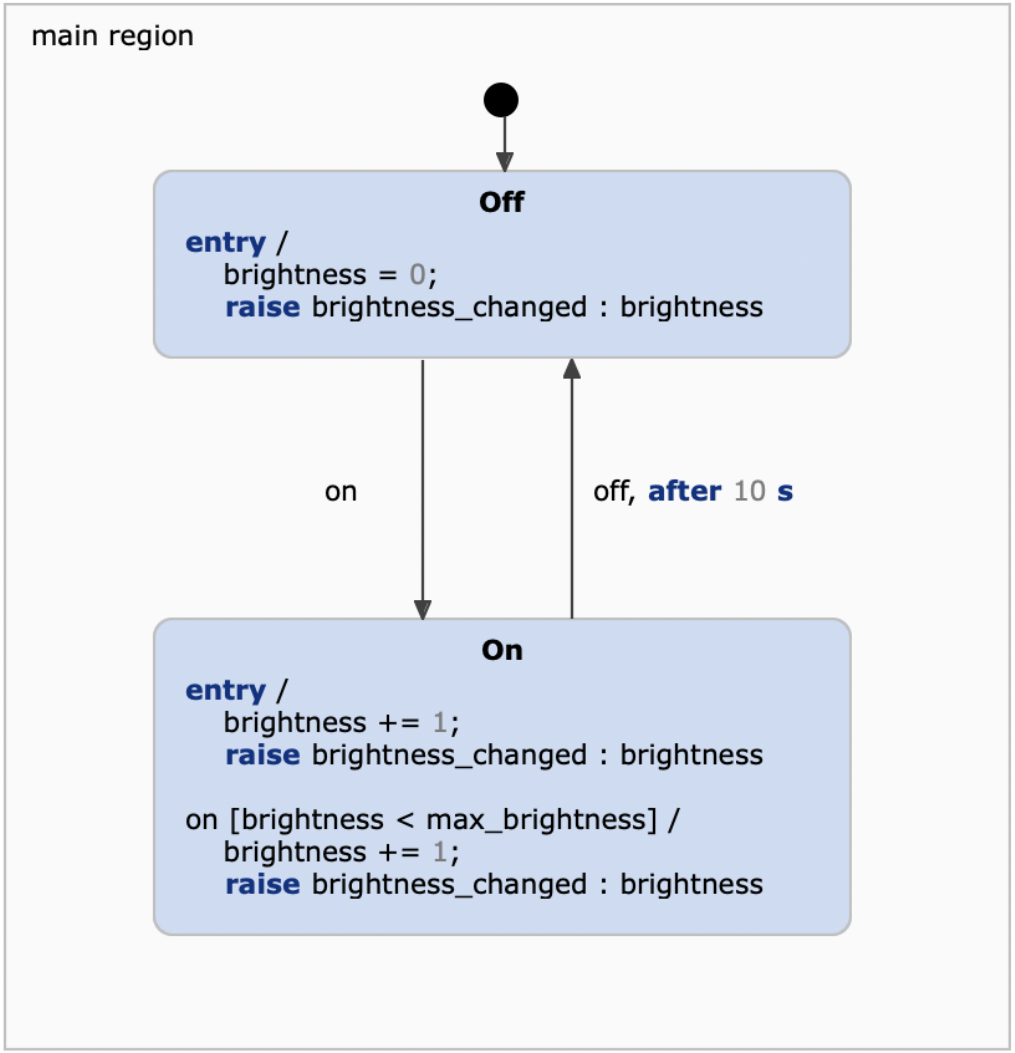
simple light switch

```
Light
@EventDriven

interface:
   var max_brightness = 10

   in event on
   in event off
   out event brightness_changed : integer

internal:
   var brightness: integer
```



Controller x 2 lights (definition section)

```
LightController
@EventDriven

// import sub machine model
import : "light.ysc"

interface user:
   in event switch_on
   in event switch_off
   in event blink_mode

interface:
   // declare sub machine variables
   var light1 : Light
   var light2 : Light

internal:
   entry /
      // enter sub machines on enter
      light1.enter;
      light2.enter;
      // set max brightness individually
      light1.max_brightness = 8;
      light2.max_brightness = 4
```

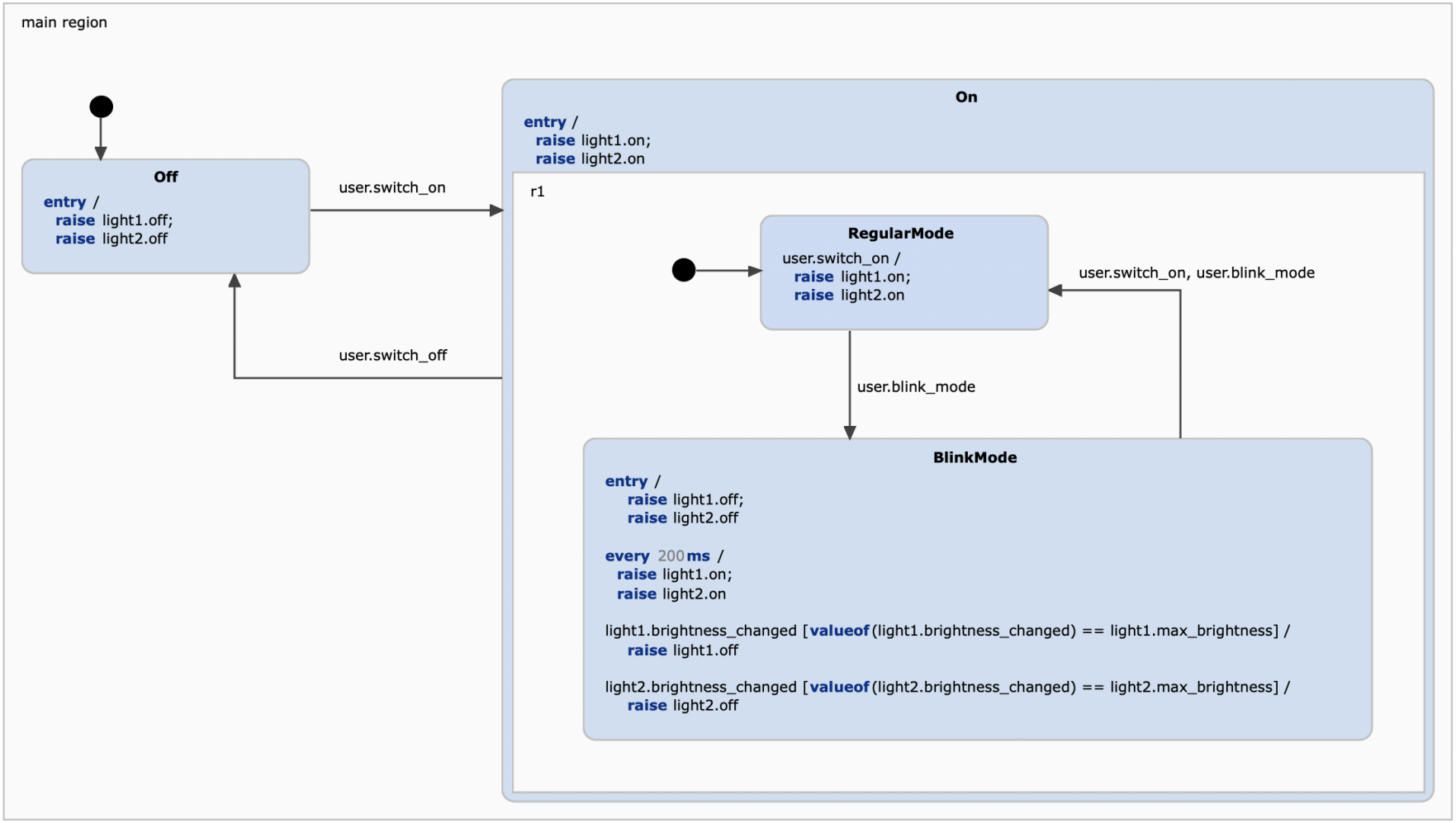
Due variabili light1 e light2 di tipo Light

- come la state machine di prima

Quando la state machine LightController inizia, anche le state machine di light1 e light2 iniziano.

Inizializzate a diversi valori di maximum brightness.

Controller x 2 lights



Esecuzione del Java code generato

```
Light 1 Brightness: 0
Light 2 Brightness: 0
Type 'On' or 'Off' to switch the lights on or off.
Type 'Blink' to toggle the blink mode.
On
Light 1 Brightness: 1
Light 2 Brightness: 1
On
Light 1 Brightness: 2
Light 2 Brightness: 2
Blink
Light 1 Brightness: 0
Light 2 Brightness: 0
Light 1 Brightness: 1
Light 2 Brightness: 1
Light 1 Brightness: 2
Light 2 Brightness: 2
Light 1 Brightness: 3
Light 2 Brightness: 3
Light 1 Brightness: 4
Light 2 Brightness: 4
Light 2 Brightness: 0
Light 1 Brightness: 5
Light 2 Brightness: 1
```

VULNERABILITÀ DEL SW

La sicurezza dei tipi nei linguaggi di programmazione

La sicurezza dei tipi in un programma è molto importante

- se l'esecutore (il PC o la macchina virtuale) non riesce a distinguere i tipi di un certo programma può facilmente causare errori
- molti attacchi sfruttano proprio debolezze nel controllo dei tipi di linguaggi diffusi come il C

"*Well-typed programs never go wrong*."

Robert Milner

Tipo

Tipo

Insieme di valori omogenei + operazioni che si possono fare (con risultato di tipo omogeneo)

Esempi:

- tipi semplici: Integers, String,
- tipi strutturati come classi, ...
- funzioni: int -> bool
  - Funzione che da un intero mi dà un boolean
  - Anche le funzioni e i metodi definisco un tipo

Esempi di non tipi:

- numeri dispari
- array contenenti String e Integer

Dipende però dal linguaggio di programmazione

A cosa servono i tipi

Per organizzare e rappresentare elementi del dominio del problema (*documentazione*)

- Spesso i tipi corrispondono ai concetti nel dominio del problema che si vuole risolvere
- L'uso dei tipi precisa cosa si vorrà fare di certi identificatori (così il compilatore può controllare)

Per istruire il compilatore come *rappresentare* i dati

- Esempio short richiedono meno bit di int

Per assicurarsi che sequenze di bit in memoria siano *interpretate* correttamente

- Per evitare errori come: 3 + true + "Angelo"

Errori di tipi: conseguenze a livello HW

✓ Confondere dati con funzioni

- Caricando quindi nei registri della CPU possibilmente codici non corretti (errore hardware)

Esempio: **x** è definita come intero e chiamo **x()**

- cerco di eseguire un dato chiamando **x()**

✓ Confondere tipi di dati semplici

Esempio: eseguo **float_add(3,4.5)** con 3 di tipo **int**
`float_add`: operazione della CPU che chiama una routine della FPU, se la CPU prende 3 come sequenza di bit float, potrebbe generare un errore hardware

Errori di tipi: conseguenze semantiche

Il programma fa qualcosa che non è quello che dovrebbe fare in base alla sua semantica

- *Esempio con tipi primitivi*: `int_add(3,4.5)`
  - In questo caso la sequenza di bit che rappresenta 4.5 è interpretato come int con perdita di valore
- *Esempio con oggetti ed ereditarietà in Java*

  Sia Quadrato sottoclasse di Figura:

  ```
  class Quadrato extends Figura
  ```

  - Considero le istanze:

    `Figura a1 = new Quadrato()` **OK**

    `Quadrato b1 = new Figura()` **NO**: Quadrato potrebbe avere dei metodi in più che potrei invocare ma non trovare perché b1 è una Figura

Type safety: sicurezza dei tipi

Un linguaggio di programmazione L si dice type safe *se non esiste programma scritto in L che possa violare la distinzione di tipi in L*

Esempi di violazioni dei tipi:

- confondere interi e float
- chiamare una funzione attraverso un intero
- accedere ad una zona di memoria sbagliata (non memory safe)

Sicurezza di alcuni linguaggi

Ecco una tabella che riporta la sicurezza di alcuni linguaggi di programmazione molto diffusi

Safety	Linguaggio	Motivo
Non safe	**C e C++**	**Type cast, aritmetica dei puntatori**
Quasi safe	**Pascal**	**Deallocazione esplicita e dangling pointers**
Safe	**Java, Python**	**Controllo completo dei tipi**

Quando si fa il type checking?

Tra i linguaggi type safe distinguiamo due categorie a seconda del momento in cui avviene il controllo dei tipi

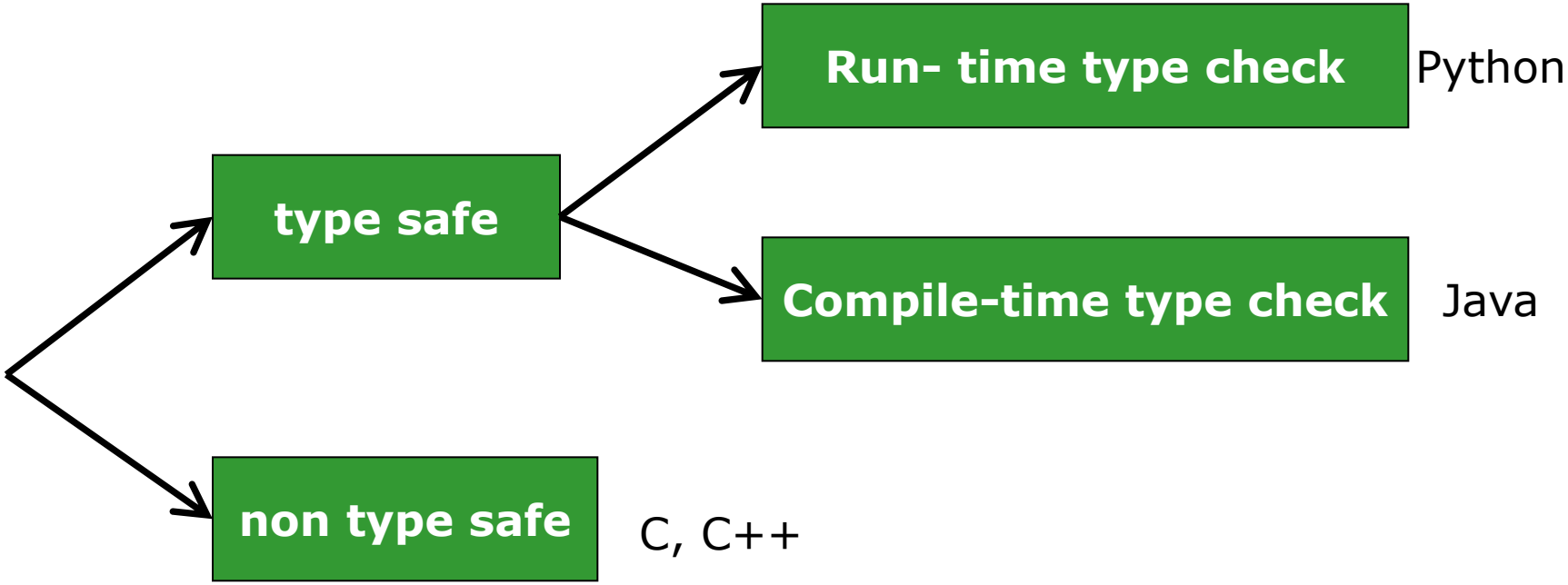
run-time type checking

- Il controllo avviene durante l'esecuzione
  - Esempio Python è un linguaggio interpretato e controllata i tipi a run-time

compile time type checking

- Il controllo avviene durante la compilazione
  - Esempio ML, Haskell, ADA sono linguaggi con meccanismi di secure static typing

Classificazione dei linguaggi



Java

Type checking avviene in tre momenti:

Static type checking

Verification Time

Run-time

Java usa compile time, però dove il compilatore non è sicuro della sicurezza dei tipi, introduce un controllo run-time

- conversioni dei tipi controllate

Java esempio

Considera la seguente sequenza di istruzioni:

```
class Quadrato extends Figura
Figura b;
Quadrato a = (Quadrato) b
```

- la conversione al sottotipo `Quadrato` è corretta solo se b è effettivamente una istanza di `Quadrato` (o di una sottoclasse)
- tale controllo non si può fare in compilazione
- il compilatore introduce un controllo da fare durante l'esecuzione che b sia convertibile a `Quadrato`

Pro e contro

Entrambi gli approcci (run-time e compile time) prevengono errori di tipo, però:

- run-time checking rallenta l'esecuzione
  - controlla le conversioni di tipo ogni volta
- compile-time checking limita la flessibilità dei programmi
  - tutte le istruzioni anche non eseguite devono essere corrette

alcuni programmi che non sono corretti compile time sono invece run time corretti

Pro e contro

alcuni programmi che non sono corretti compile time sono invece run time corretti

<u>esempio</u>

In Java, non posso scrivere

```
int x;
if (0>-1) { x++;} else { x = "ciao";} NO
```

perchè assegna ad x int una String

eppure questo programma è runtime type safe, perchè nessuna esecuzione causa errori di tipo (0 è sempre > -1)

“C is not Safe”

Alcune caratteristiche del linguaggio C e C++ che possono dare errori:

1. dereferenziazione del null
2. type cast non controllato
3. pointer arithmetic
4. accesso alla memoria non valida
    1. violazione spaziale come out of bound
    2. violazione temporale come dangling pointers

Queste caratteristiche rendono il C molto flessibile e veloce a discapito della sua sicurezza

- è responsabilità del programmatore stare attento a non introdurre difetti

Dereferenziazione di null

- **Deferenziazione= operazione di accedere alla memoria tramite puntatore**
- La dereferenziazione di un puntatore in C non viene controllata
  - Se accedo ad una cella puntata da un puntatore nullo ho "segmentation fault", cioè un errore del sistema operativo

Programma con accesso tramite puntatore null

```
int main(){
    int * ptr;
    ptr = 0;
    *ptr = 2;
}
```

TypeCast non Safe

Il C permette la conversione non controllata da un tipo ad un altro:

- da un tipo ad un sopratipo con possibile perdita di informazioni
- da intero ad una funzione per cercare di eseguire una certa locazione di memoria che potrebbe non essere un'istruzione corretta o fare qualcosa di non voluto

Programma corretto in C ma con type cast non safe:

```
double d;
int i;
…
i = d;
```

➔ possibile perdita di informazioni

Pointer arithmetic

In C i puntatori sono trattati come interi.

Mediante l'aritmetica dei puntatori possiamo accedere a zone di memoria con tipi diversi.

Esempio:

```
// p è di tipo A* (int*). ATTENZIONE: i dati a p+2 NON
sono un 'A' ma i byte del double.
    A *p = (A*)buf;
    A x = *(p + 2);   // L'espressione ha tipo A, ma
legge i byte del double
```

- se il puntatore `p` è definito di tipo `A*`, l'espressione `*(p+i)` ha tipo `A`
- però il valore memorizzato a `p+i` potrebbe avere qualsiasi tipo
- l'assegnamento `x = *(p+i)` con `x` di tipo `A`, permette di memorizzare in `x` un valore di qualsiasi tipo (quello in `*(p+i)` ma senza controllo che sia di tipo `A`)

C non è memory safe: violazione memoria (1)

Mediante i puntatori si può facilmente accedere alla memoria in modo scorretto

```
void f(int* p, int i, int v) {
    p[i] = v;
}
```

Accedo in questo modo all'indirizzo p+i e p+i potrebbe contenere dati importanti o altro codice

- Posso modificare il return address di una chiamata di una procedura ed eseguire altro codice, posso modificare dei diritti o leggere informazioni riservate
- Tipico "buffer overflow"

C non è memory safe: violazione memoria (1)

Mediante i puntatori si può facilmente accedere alla memoria in modo scorretto

```
void f(int* p, int i, int v) {
    p[i] = v;
}
```

Accedo in questo modo all'indirizzo p+i e p+i potrebbe contenere dati importanti o altro codice

- Posso modificare il return address di una chiamata di una procedura ed eseguire altro codice, posso modificare dei diritti o leggere informazioni riservate
- Tipico "buffer overflow"

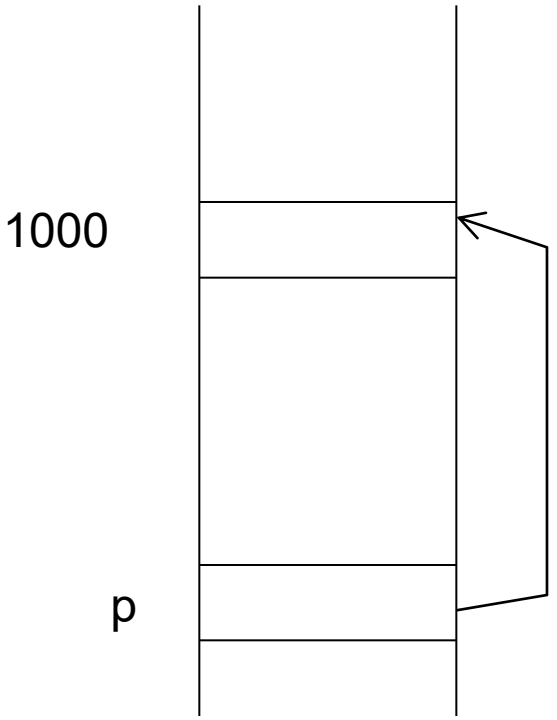
C non è memory safe: violazione memoria (2)

I puntatori in C sono assimilati a interi

Tramite cast di dati interi a puntatori, posso accedere ad una zona di memoria a piacere

*Programma (OK in compilazione) con conversione da int a char**

```
int main(){
  char* p;
  p = 1000;
  *p = 'a';
}
```



Deallocazione esplicita e Dangling Pointers

In C una locazione puntata da un puntatore *p* può essere deallocata (liberata) dal programmatore: p è un "dangling pointer"

Un puntatore è dangling se punta ad una zona di memoria che è stata liberata per essere riutilizzata

- Il sistema operativo potrebbe allocare la stessa memoria nuovamente per memorizzare un altro tipo di valore
- Posso continuare ad usare *p* per accedere a questa memoria e rompere la type safety

Dangling Pointers sullo stack

Un esempio frequente di errore dovuto a dangling pointers è quando si usano puntatori a celle dello stack

Si verifica quando:

- si crea un puntatore *p* ad una zona *A* di memoria che è locale ad un metodo (ad esempio variabili locali)
- *A* è quindi allocata sullo stack
- *A* viene liberata all'uscita del metodo
- *p* è a questo punto un dangling pointer

Esempio

Funzione che converte un intero in stringa corrispondente, restituendo il puntatore alla stringa ottenuta:

```
char* itoa(int i){
      char buf[20];
      sprintf(buf,”%d”,i);
      return buf;
}
```

- A cosa punta buf? buf viene restituito ma punta ad un array locale che viene deallocato
- buf è un dangling pointer

Soluzione

Come si possono evitare dangling pointers?

1. Evitare di puntare zone di memoria sullo stack ed usare la malloc:

```
struct Point * result =
                malloc(sizeof(struct Point))
```

   La malloc crea puntatori a zone sicure

2. Uso del garbage collector (gc) invece che della deallocazione esplicita
   - Il gcc marca lui le zone da liberare e che si possono riutilizzare

Cosa fare per avere evitare tali errori?

Se vogliamo scrivere codice safe cosa possiamo fare?

- Scrivere attentamente, progettare prima, documentare, etc.

Se vogliamo essere sicuri che il nostro codice è safe?

- Due soluzioni possibili
  1. usare linguaggi type safe (Java, Python;..) e linguaggi + astratti
  2. usare linguaggi come C e dei tools che ci aiutano a rendere i programmi C safe

Svantaggi ad usare linguaggi più astratti

C'è un prezzo da pagare se si vogliono usare linguaggi safe come Java, ...

1. **Prestazioni inferiori**
   - per il Controllo dei limiti nell'accesso agli array, garbage collection per evitare dangling pointers
2. **Impiego di maggiore memoria**
   - per tenere informazione sui tipi, sulla dimensione degli array
3. **Annotazione dei tipi**
   - maggiore verbosità nelle dichiarazioni
4. **Porting di codice già esistente in C**
   - (per quanto Java abbia sintassi simile al C)

Vantaggi del C

Il C è tutt'oggi usato per molte applicazioni come il sistema operativo, i device drivers

- Ha prestazioni elevate
- Permette la gestione esplicita della memoria
- Permette il controllo della rappresentazione dei dati a basso livello
- Riuso del codice esistente già scritto in C

Alcune tipiche vulnerabilità del C

Possono causare problemi di sicurezza:

Errori "spaziali" di accesso alla memoria

- Out of bound access, buffer overflow, …

Errori "temporali" di accesso alla memoria

- Dangling pointers

Errori di cast

- Tra diversi tipi di puntatori, tra puntatori e dati interi, tipi unione

Memory leaks

- Programmi che non rilasciano la memoria anche quando non serve più

Come rendere il C safe?

1. Tools per l'analisi statica e dinamica per trovare safety violations

- esempio **Purify** della Rational/IBM è un tool per l'analisi dinamica per scoprire errori di accesso alla memoria

2. Librerie per rendere il programmi C safe

3. Tools, e linguaggi per prevenire safety violation con due approcci distinti

a) rendere sicuri programmi C : SafeC, CCured

b) varianti safe del C: **Cyclone**, Vault

Analisi dinamica con Purify

approccio analisi dinamica

input programmi C/C++ (qualsiasi)

output eseguibili linkati con Purify

metodo inserimento di controlli per trovare durante l'esecuzione errori di accesso alla memoria o memoria non rilasciata

pro si applica a codice già esistente

contro rallenta l'esecuzione e non garantisce la scoperta di ogni errore

info http://www-306.ibm.com/software/awdtools/purify/

valutazione: ★★★

Cyclone

approccio safe C-like language

input programmi C modificati

output programmi C sicuri

metodo controlli dinamici solo dove necessario e garbage collector

pro minimo overhead di tempo e di memoria

contro richiede di modificare i programmi originali

info http://cyclone.thelanguage.org/

valutazione: ★★★

JAVA SANDBOX

Bibliografia

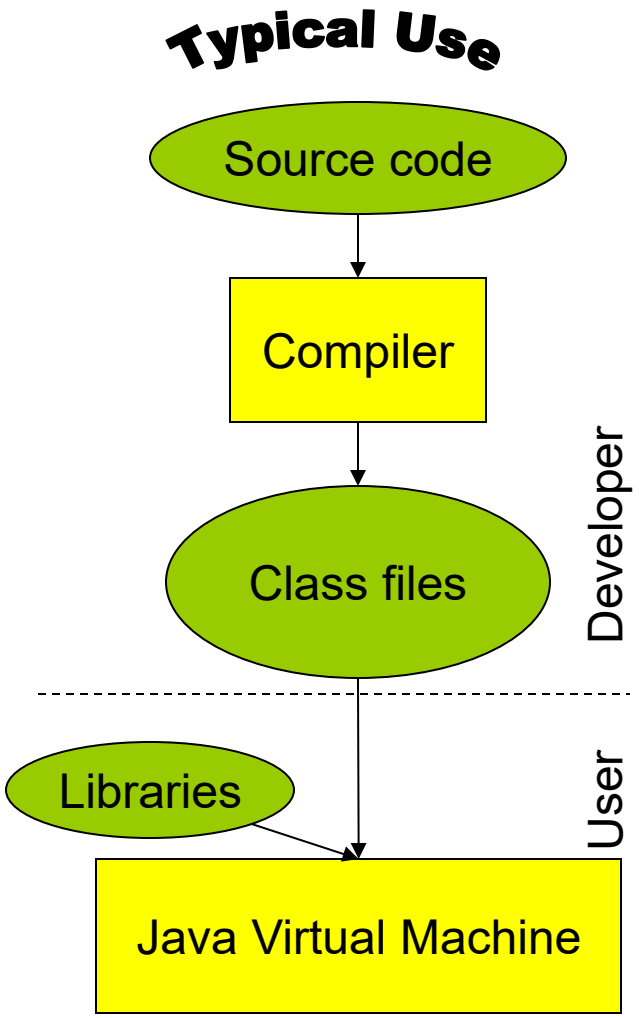
Li Gong. *Java™2 Platform Security Architecture. 1999*

Li Gong, Gary Ellison, Mary Dageforde. Inside Java™ 2 Platform Security: Architecture, API Design, and Implementation, Second Edition. Addison Wesley. 2003. ISBN: 0-201-78791-1

Alfredo De Sanctis. Il Modello di sicurezza in Java (su ariel). 2 documenti

Java e JVM

- **source code**
  - `prova.java`
- **compiler**
  - `javac prova.java`
  - produce `prova.class`
- **JVM**
  - `java prova.class`
- Si può anche solo usare la JVM



Meccanismi di sicurezza di Java

Questa architettura sicura si basa su numerosi meccanismi:

- **sicurezza dei tipi**
  - Java è sicuro per quanto riguarda i tipi
- **verifica del bytecode (Java compilato)**
  - solo codice Java legittimo viene accettato
- **controllo runtime delle classi caricate**
  - la Java virtual machine (JVM) carica e controlla il codice dinamicamente
- **accesso mediato alla risorse**
  - i programmi accedono attraverso la JVM

Diverse politiche di sicurezza

- **Modello Java 1.0/1.1**
- **Modello da Java 2**

Applets e Applicazioni

Nel modello di sicurezza di Java 1.0:

applets:

- ogni codice che viene eseguito scaricandolo da Internet
- si assume che sia malizioso
- viene eseguito in un sandbox
  - ambiente isolato e controllato

applicazioni:

- programmi sul filesystem locale
- assimilabili ai normali programmi

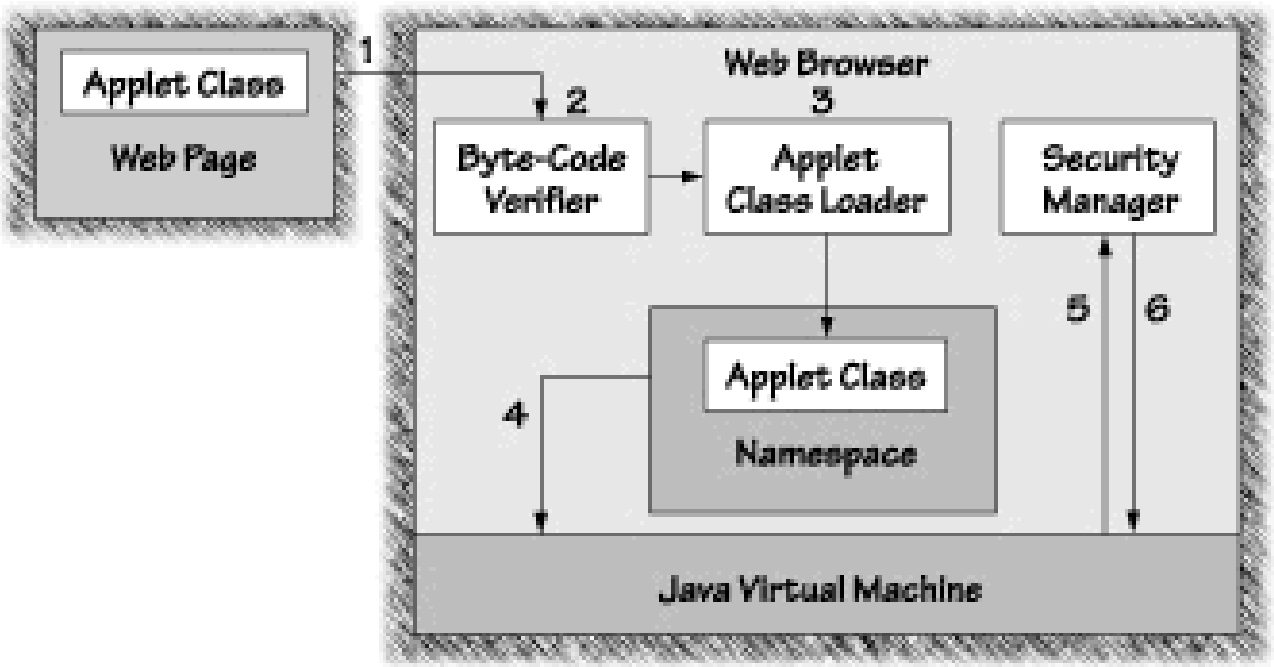
Java 1.0 Security Policy

Sandbox Policy per un applet:

- non può accedere direttamente al file system locale o ai devices
- può connettersi solo con la sorgente da cui è proviene l'applet
- non può eseguire nessun programma locale
- viene marcato come **`untrusted`** e viene visualizzata un icona
- non può manipolare (estendere, …) la classi base

Le applicazioni hanno accesso illimitato

Architettura della sandbox



Bytecode verifier

Il compilatore controlla già che il sorgente non violi le regole di sicurezza di Java, ma non è detto che il byte code sia prodotto dal compilatore !!!

Il bytecode prima di essere eseguito viene verificato per controllare che:

- non esegue conversioni illegali tra dati
- non modifica (forge) puntatori (riferimenti)
- non viola le restrizioni di accesso
  - es: accede ad un campo privato
- usa in modo corretto gli oggetti per come dichiarati
- chiama i metodi in modo corretto (argomenti)

Efficienza

Nota che la verifica del bytecode viene fatta staticamente

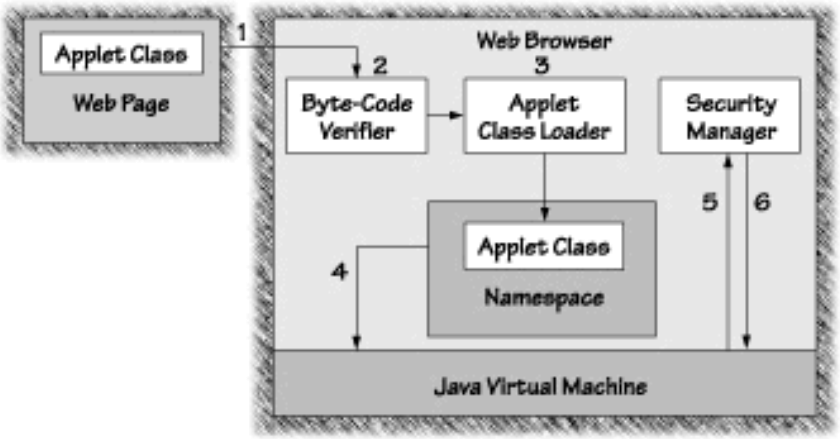
Gli stessi controlli potrebbero essere fatti sempre runtime dalla JVM

- rallenterebbe l'esecuzione
  - Ad esempio il controllo di un metodo andrebbe fatto ogni volta che è invocato
- durante l'esecuzione la JVM può assumere che alcuni controlli siano già stati eseguiti

Security Manager e ClassLoader

- **In JDK 1.0, la sandbox è costituita, oltre che dal Bytecode Verifier, dalle due classi**

  - SecurityManager
  - ClassLoader



I 3 componenti lavorano insieme per garantire la sicurezza.

SecurityManager

Classe che definisce i metodi di controllo dei permessi di sicurezza chiamati dal sistema

- esempio: “checkRead(String filename)”
- se non è permesso lancia un'eccezione

In Java 1 è possibile creare una propria security policy:

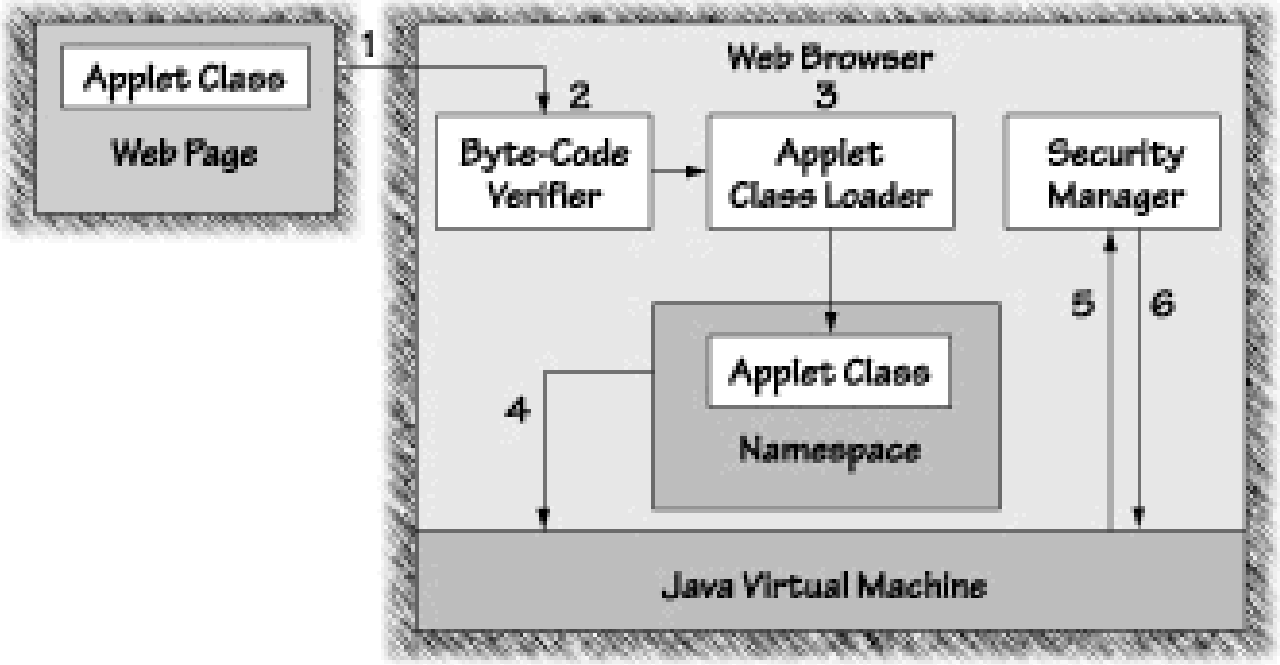
- crea una sottoclasse di SecurityManager
- installa un'istanza con *System.setSecurityManager*;
  - nota che una volta settata non può essere revocata o cambiata

ClassLoader

Carica effettivamente le classi:

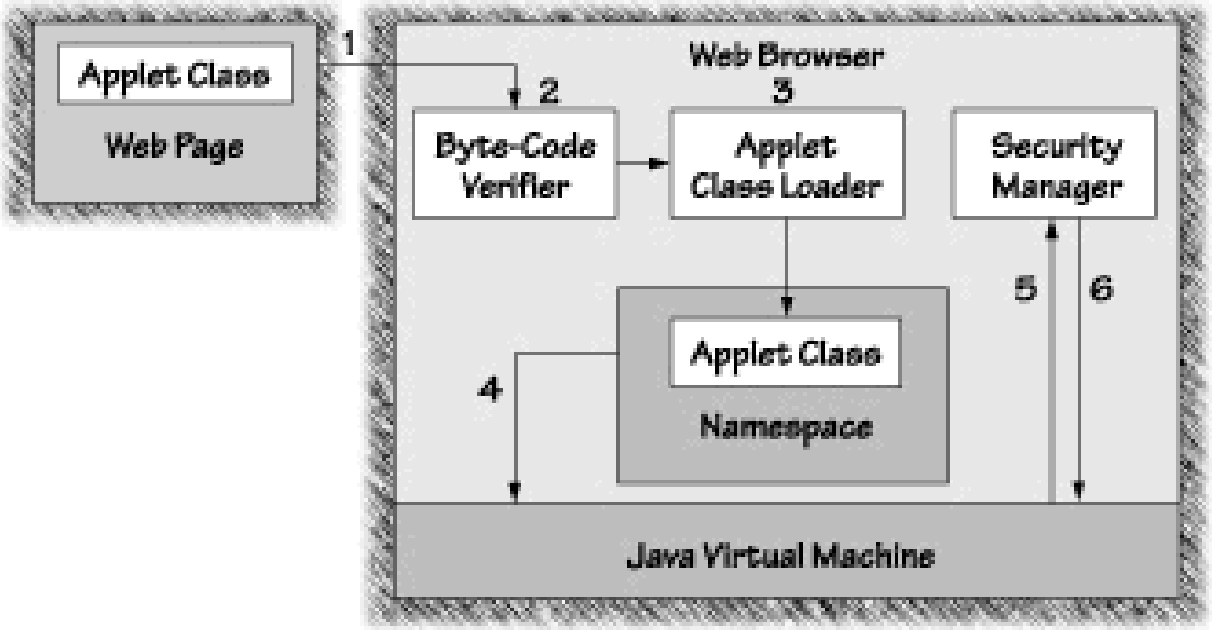
- dato il nome della classe trova la sua definizione ed istanzia una classe o insieme di classi in un *namespace*
- ogni classe mantiene un riferimento all'istanza di class loader che l'ha caricata
- mantiene i namespace di applet diverse separati (mediante diverse istanze di ClassLoader)
- le classi nel "CLASSPATH" sono classi fidate (in JDK 1.0-1.1), classi di sistema sempre fidate, altrimenti invoca il bytecode verifier e SecurityManager per i permessi

Approccio di sandbox



- Il codice sorgente Java è compilato in un Java bytecode che è trasmesso attraverso il Web al browser che lo richiede.
- L'HTML, in una pagina Web, specifica quale codice deve essere preso dal Web server.
- La richiesta del browser Web, indotta da un'azione quando un utente clicca su un hyperlink, …

ClassLoader



(1) preleva il bytecode dal Web, (2) lo verifica,

(3) lo istanzia in una classe o insieme di classi in un namespace. L'applet è eseguito: quando un metodo viene invocato

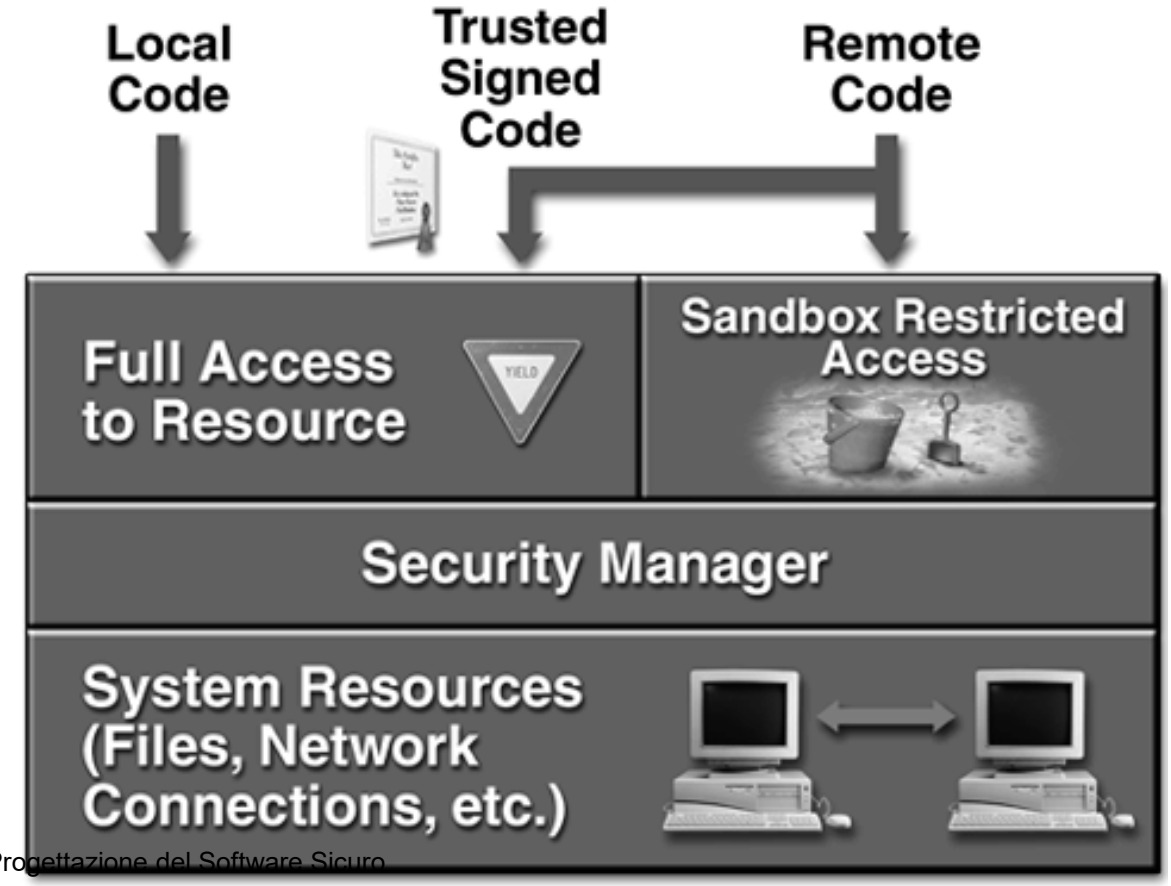
(4) il ClassLoader trova il riferimento alla classe e se il metodo è pericoloso

(5) il Security Manager viene consultato prima che il metodo venga eseguito. Il Security Manager

(6) esegue i runtime-checks sulla base dell'origine della classe chiamante e con l'autorità di porre un veto sull'operazione.

Applet firmate in JDK 1.1

JDK 1.1 introduce il concetto di applet firmate digitalmente che hanno i permessi delle applicazioni



Java 2

Java 2 introduce il controllo degli accessi a livello più fine basato su

- security policies
- permessi

Gestisce la politica flexible access control e fine-grained access control : supera il modello "tutto" o "niente" di JDK 1

- alcune applet possono avere alcuni diritti per accedere ad alcune risorse fuori dal sandbox
- **Esempio:** una applet può leggere un certo file in cui memorizzo i dati ma nessun altro

Esempio di Security Policy

Origine del codice (CodeSource)		Permissions
Base URL	Signature	
http://www.schwab.com/classes/stockeditor.jar	Schwab's signature	•Read/write file /home/daw/stocks
http://*.schwab.com/	(not required)	•Connect/accept bankofamerica.com ports 1-1023 •Read file /home/daw/logo.png

CodeSource e Permission

La classe CodeSource specifica da dove proviene il codice:

- l'URL da cui proviene la classe
- un array di certificati

È immutabile

La classe Permission rappresenta, con la sua gerarchia, i permessi

- *FilePermission* permessi di scrittura/lettura di file
- *SocketPermission* per connettersi ai socket
- …

Vantaggi dell'architettura di Java 2

- **Non è necessario scrivere codice particolare per modificare la SecurityPolicy**
- **Separazione tra espressione della Security Policy e la sua applicazione:**
  - l'*espressione* dei diritti di una applicazione avviene usando istanze della classe Permission e delle sue sottoclassi
  - l'*applicazione* della security policy è fatta dal SecurityManager con un metodo generale checkPermission (e da AccessController)

ProtectionDomain Class

Un ProtectionDomain

- viene creato da un CodeSource e una PermissionCollection (insieme di Permission)
- definisce l'insieme di permessi dati alla classe
- ogni classe appartiene ad una istanza ProtectionDomain, impostata al momento della creazione e che non può essere cambiata
- l'accesso a questi oggetti è ristretto

Origine del codice (CodeSource)		Permissions
Base URL	Signature	
http://www.schwab.com/classes/stockeditor.jar	Schwab's signature	•Read/write file /home/daw/stocks
http://*.schwab.com/	(not required)	•Connect/accept bankofamerica.com ports 1-1023 •Read file /home/daw/logo.png

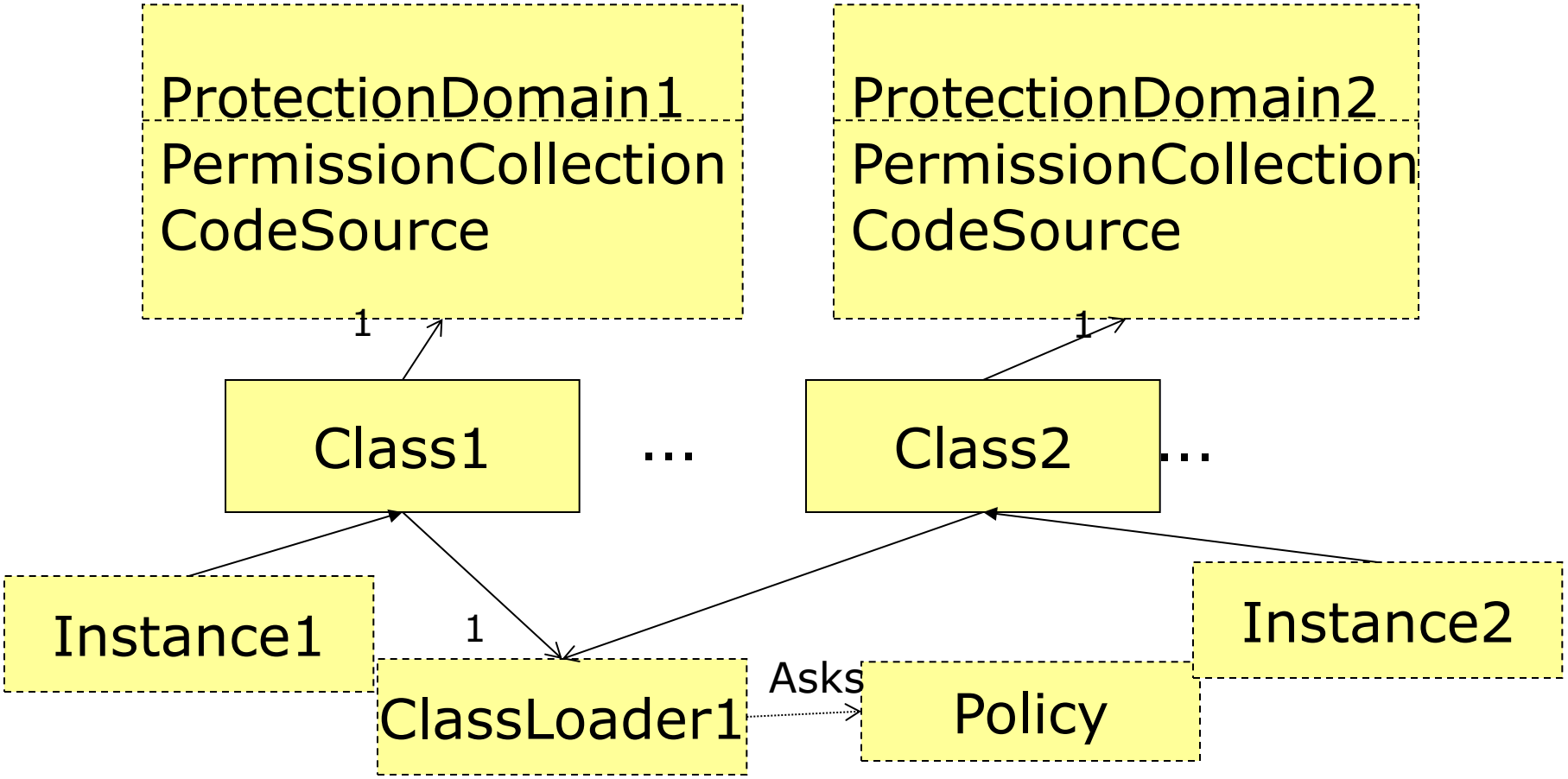
Policy Class

E' l'interfaccia per definire politiche di sicurezza particolari

- dato un CodeSource, restituisce un PermissionCollection
- viene usata durante il caricamento di una classe (invocata da ClassLoader) per impostare il ProtectionDomain

Origine del codice (CodeSource)		Permissions
Base URL	Signature	
http://www.schwab.com/ classes/stockeditor.jar	Schwab's signature	•Read/write file /home/daw/stocks
http://*.schwab.com/	(not required)	•Connect/accept bankofamerica.com ports 1-1023 •Read file /home/daw/logo.png

Impostazione del ProtectionDomain



Caricamento di una classe

Quando una classe C1 richiede di caricare (load) una classe C2 non ancora caricata

1. Il ClassLoader di C1 cerca il C2.class, lo carica e chiama il bytecode verifier

3. Il CodeSource di C2 viene determinato

4. L'oggetto Policy del ClassLoader restituisce le Permissions dato il CodeSource di C2

5. Se un ProtectionDomain già esistente ha lo stesso CodeSource e Permissions, viene riusato, altrimenti viene creato un nuovo ProtectionDomain; C2 viene assegnata al suo ProtectionDomain

la classe può essere istanziata e i suoi metodi eseguiti, i controlli runtime continuano ...

Controllo run-time della sicurezza

- **Se una classe C invoca il metodo M**
- **metodo M richiede il permesso P**
  - la JVM chiama il checkPermission(P) del SecurityManager corrente
- **questo attiva un "AccessController" che controlla il chiamante di M, cioè C**
  - se il ProtectionDomain di C contiene il permesso P allora va bene, altrimenti solleva un'eccezione
    - controlla tutte le chiamate di metodi nella catena delle chiamate